CTF Challenge — Guardian Gap Hunt (Demo-3 Three-Tier Edition)

DEFCON Coimbatore 2026 · Venugopal Parameswara

Format: Capture The Flag | **Duration:** 45–120 minutes per level **Prerequisite:** Demo-3 running locally (`./setup_and_run3.sh start`)

> *"Every system has a gap. Demo-3 has five tiers — so it has at least five gaps."*

Why Demo-3 Has More Gaps Than Demo-2

Demo-2 had one tier and two gaps (existence-not-integrity check; append-injection). Demo-3 has **five honest gaps** already documented in §9 of `demo3_technical.md`. This CTF treats those gaps as starting points and dares participants to find more.

This is not a trick — the talk openly discloses every gap. The CTF asks participants to:

1. **Reproduce** each disclosed gap (prove it exists, not just read about it)
2. **Quantify** each gap (how many rows leak? how fast? under what load?)
3. **Close** each gap (patch guardian, write a CRD, tighten the poll loop)

Setup

```
cd /Users/kuttanadan/Documents/defcon-demo/Demo-3
./setup_and_run3.sh setup     # once
./setup_and_run3.sh start
./setup_and_run3.sh check     # expect 5 × (healthy)
```

Verify the JSONL log is being produced:

```
tail -f /tmp/demo3_guardian.jsonl | head -5
```

Level 1 — Reproduce the Bulk-Insert Leak 🟢

Points: 100 | **Difficulty:** Easy–Medium

Objective

§9 of `demo3_technical.md` claims:

> *"~500 rows still reach the replica before `pg_wal_replay_pause()` engages."*

Prove it. Run the DB attack, count rows that made it to the replica before the fence, and report the exact number.

Hints

```
# Baseline replica row count
docker exec demo3-db-rep psql -U phoenix -d phoenix -t -c \
  "SELECT COUNT(*) FROM transactions WHERE amount < 0;"

# Trigger the attack
./attack3_db.sh

# During fence window (check with this)
docker exec demo3-db-rep psql -U phoenix -c "SELECT pg_is_wal_replay_paused();"

# Row count on replica DURING the fence (this is your leak measurement)
docker exec demo3-db-rep psql -U phoenix -d phoenix -t -c \
  "SELECT COUNT(*) FROM transactions WHERE amount < 0;"
```

Submission Requirements

1. The exact count of negative-amount rows on the replica during the fence window
2. Terminal output showing `pg_is_wal_replay_paused()` returned `t`
3. A one-paragraph explanation of WHY rows leaked — i.e., what happens between the first UPDATE wave and guardian's next 1-second poll cycle

Flag Format

`DEFCON_CBR_D3{level=1, leak_rows=<N>, cause=<bulk_insert_before_poll>}`

Example

`DEFCON_CBR_D3{level=1, leak_rows=487, cause=bulk_insert_lands_inside_single_poll_tick}`

Level 2 — Close the DB Gap 🟡

Points: 300 | **Difficulty:** Medium–Hard

Objective

Modify the detection path so that the leak from Level 1 drops **below 100 rows**. The leak doesn't have to reach zero — but it has to be meaningfully smaller.

Valid Approaches

Any of these is acceptable (more are welcome — grade on measured reduction):

1. **Tighten the poll** — reduce `POLL_INTERVAL` in `guardian3.sh`. Measure the leak reduction. What's the CPU cost?
2. **Lower the threshold** — the current `delta > 500` is the trigger. Try 200, 100, 50. Where does it false-positive on normal batch INSERT jobs?
3. **Trigger-based detection** — add a PostgreSQL trigger on `transactions` that increments a counter on each INSERT/UPDATE. Guardian reads the counter instead of `pg_stat_user_tables`. What's the write-amplification cost?
4. **Logical replication + `pg_logical_emit_message`** — use a logical slot + decoder to get per-row signals. What's the infrastructure cost?

Measurement

Run `attack3_db.sh` five times with your modification and report:

- Leak rows: min / avg / max
- False positives during a simulated legitimate workload (seed 1000 rows in a tight loop without mass UPDATE)
- Observed P95 CPU of `demo3-db-pri` during the test window

Submission Requirements

1. Diff of your `guardian3.sh` (or a new file if you took the trigger/CDC approach)
2. Table of 5 runs showing leak count and false-positive count
3. Brief written explanation of the trade-off you chose

Flag Format

`DEFCON_CBR_D3{level=2, approach=<tighter_poll|trigger|cdc|other>, avg_leak=<N>, false_pos=<N>}`

Level 3 — Patch the Fence-to-Promote Gap 🔴

Points: 500 | **Difficulty:** Hard

Objective

§9 also states:

> *"DB recovery is fence-and-resume, not full failover. Full failover via `pg_ctl promote` is intentionally not invoked automatically."*

Your job: **make it automatic, safely.** The existing `db_promote_replica` function in `guardian/guardian3.sh` is wired but never called. Write the conditions under which it *should* be called, add the invocation, and prove it works end-to-end.

The Hard Part

Automatic failover is dangerous. Two common failure modes:

- **Split-brain:** primary comes back, now you have two writers
- **Premature promote:** you promote the replica for a transient glitch, and the primary's ahead-of-replica data is lost

A good solution addresses both.

Hints

```
# Inspect the existing (dormant) promote function:
grep -A 30 "db_promote_replica" /Users/kuttanadan/Documents/defcon-demo/Demo-3/g

# What signals could you use to decide "promote vs fence-and-resume"?
#   - Primary unreachable for > N seconds
#   - Primary WAL hasn't advanced for > M seconds
#   - Health check fails on both endpoints and replica WAL is healthy
#   - External consensus signal (etcd, Patroni-style DCS)
```

Submission Requirements

1. Patched `guardian3.sh` with:
   - Clearly named promote-decision conditions
   - A "STONITH-equivalent" step that prevents split-brain (even if symbolic)
   - A brief comment block describing the recovery runbook
2. Terminal log of a run where:
   - Primary is killed (`docker kill demo3-db-pri`)
   - Replica is promoted automatically
   - A new transaction writes successfully to the new primary
3. A written rollback procedure — if you get this wrong in production, how do you recover?

Flag Format

`DEFCON_CBR_D3{level=3, promote_trigger=<condition>, split_brain_guard=<mechanism>, test_mttr_ms=<N>}`

Level 4 — The Supply-Chain Attack (Expert) 🟣

Points: 600 | **Difficulty:** Expert

Scenario

You cannot touch the running containers — guardian kills your shell in under a second. Instead, you poison the **golden image itself**.

Objective

Modify the build context so that the next `./setup_and_run3.sh setup` builds a compromised golden image — one that looks identical to the legitimate one but contains a backdoor.

Constraints

- The poison must survive `docker rmi demo3-web:golden && docker build …`
- The running container must still serve HTTP 200 on `/health`
- Guardian must not detect any drift from the poisoned golden image (because it *is* the golden image now)

The Three Deliverables

This level is 50% exploit, 50% defence. Submit all three:

1. **Attack** — A minimal, documented supply-chain poison. Examples:
   - Dockerfile addition that fetches a remote script at build time
   - A backdoored base image (build a local `nginx:alpine` variant and retag)
   - A `RUN` line that writes a cron entry or a delayed payload
2. **Detection** — Show at least two ways the poison *could* have been caught:
   - `cosign verify` failing on the signed image
   - SBOM diff (Syft) showing an unexpected package
   - Layer-diff on the Dockerfile against a known-good commit
   - Image-scanner alert (Trivy/Grype)
3. **Prevention** — Describe (≤ 1 page) the production pipeline that would have prevented your attack:
   - Signed base images
   - SLSA provenance attestations
   - Admission controller rejecting unsigned images
   - Registry with binary content-addressable storage

Flag Format

`DEFCON_CBR_D3{level=4, vector=<dockerfile|base_image|dep|build_script>, detection=<cosign|sbom|layer_diff|scanner>, prevention=<slsa|admission|signed_registry>}`

Bonus Challenge — The WAF 14%

Points: 200 | **Difficulty:** Medium

§9 claims the WAF blocks 13/15 = 86% — honest because one request returned 404 and the "bypass" had no real target. The 14% miss is classified as PASSED.

Challenge: Without disabling ModSecurity, construct **one additional request** that:

- Returns 200 from `demo3-waf` (actually reaches `demo3-web`)
- Carries a payload that, *if* `demo3-web` had a vulnerable route, would exploit it
- Would genuinely count as a "miss" that the team should fix — not a 404

Hints

- ModSecurity CRS at paranoia level 1 is deliberately permissive
- Look at the `nginx-waf.conf` to see what routes are proxied vs served locally
- Consider Unicode normalisation, double-encoding, HTTP/2 request smuggling as categories — but don't enable HTTP/2 in the demo
- Some CRS rules only fire on specific Content-Types

Submission

1. The exact `curl` command
2. The HTTP response code from the WAF
3. The ModSecurity audit log entry (should show rule evaluation — even if pass)
4. A written paragraph explaining what the real exploit would be if a vulnerable handler existed

Flag Format

`DEFCON_CBR_D3{bonus=waf_bypass, category=<normalisation|encoding|header|content_type>}`

Bonus Challenge — The Dashboard Injection

Points: 150 | **Difficulty:** Medium

The AI agent's `/trigger/{target}` endpoint has an allow-list: `{web, app, db, waf}`. Unknown targets return 400.

Challenge: Without modifying `ai_agent3.py`, can you trigger any attack script that isn't in the allow-list? Or, alternatively, find a way to pass arguments to one of the four allowed scripts that changes its behaviour?

Hints

- Read the `Handler.do_GET` method in `ai_agent3.py`
- The `subprocess.Popen` call uses a fixed argument list — but the environment, cwd, and script content are still in play
- What if an attack script `source`s something? What if one of them reads a config file?

Submission

1. The exact HTTP request used
2. Evidence that a non-listed or modified-behaviour script ran
3. A proposed code fix in one line or less

Flag Format

`DEFCON_CBR_D3{bonus=trigger_escape, vector=<env|cwd|script_source|other>}`

Hints & Reference

Useful Commands

```
# Watch all guardian events live
tail -f /tmp/demo3_guardian.jsonl | python3 -c "
import sys, json
for line in sys.stdin:
    try: d=json.loads(line); print(f'{d[\"ts\"]} [{d[\"tier\"]:7}] {d[\"event\"]
    except: pass
"

# Dashboard state snapshot
curl -s http://localhost:7880/state | python3 -m json.tool

# All WAF audit entries from the last run
docker exec demo3-waf tail -200 /var/log/modsec/audit.log

# Postgres replication status
docker exec demo3-db-pri psql -U phoenix -d phoenix -c \
  "SELECT client_addr, state, sent_lsn, replay_lsn,
          (sent_lsn - replay_lsn) AS lag FROM pg_stat_replication;"

# Replica's replay state
docker exec demo3-db-rep psql -U phoenix -c \
  "SELECT pg_is_wal_replay_paused(), pg_last_wal_replay_lsn();"
```

What Guardian Actually Checks

```
WEB:  docker exec test -f /usr/share/nginx/html/index.html
      curl -s localhost:9091/health = 200?
      → on 2 consecutive failures: respawn

APP:  curl -s localhost:3001/health = 200?
      docker ps --filter name=demo3-app running?
      → on 2 consecutive failures: respawn

DB:   pg_stat_replication max(sent_lsn - replay_lsn) > 1 MB? → log warning
      Σ(n_tup_ins + n_tup_upd + n_tup_del) delta > 500/sec? → fence replica
      10 quiet ticks after fence → resume replica, emit REPLICA_RESUMED
```

What Guardian Does NOT Check

```
✗ File content (no checksum — Level 1 gap carried over from Demo-2)
✗ Image signature (supply-chain gap — Level 4)
✗ Process list inside containers
✗ Network egress from containers
✗ Schema changes (CREATE/DROP TABLE doesn't trip the write-velocity counter)
✗ Slow-and-low write patterns below 500/sec threshold
✗ Primary down (the promote path is dormant — Level 3)
✗ Replication slot health (if a slot falls behind, no signal)
✗ Container escape (T1611)
```

Scoring Summary

Level	Challenge	Max Points
Level 1	Reproduce Bulk-Insert Leak	100
Level 2	Close the DB Gap	300
Level 3	Patch the Fence-to-Promote Gap	500
Level 4	Supply-Chain Attack	600
Bonus	WAF 14%	200
Bonus	Dashboard Injection	150
Total		**1,850**

Suggested Prizes

- 🥇 First team to complete Level 3: DEFCON badge + speaker shout-out
- 🥈 Best-documented Level 4: printed Demo-3 stack diagram signed by the speaker
- 🥉 Most elegant Level 2 solution: featured in the next DEFCON talk

---

Instructor Answer Key

Level 1

Expected `leak_rows` ≈ 400–500 on the M5 reference machine. The cause is that the 5 UPDATE waves each do `UPDATE … WHERE id IN (SELECT id FROM transactions ORDER BY RANDOM() LIMIT 200)`, producing ~200 row-changes per wave. With a 0.8s sleep between waves, wave 1 completes entirely before guardian's next poll tick sees the delta. Roughly 2–3 waves (~400–600 rows) propagate to the replica before `pg_wal_replay_pause()` fires.

Level 2 — Reference solution

Reducing `POLL_INTERVAL=0.2` cuts the leak to ~100 rows but raises CPU on both `demo3-db-pri` and guardian's host process. The trigger-based approach (a `BEFORE UPDATE` row-count trigger that `NOTIFY`s a channel guardian subscribes to via `psql --wait`) gets leak close to 0 but adds ~15% write amplification.

Level 3 — Reference approach

Two signals required:

- Primary unreachable: `docker inspect --format='{{.State.Health.Status}}' demo3-db-pri` ≠ healthy for 10s
- Primary WAL stalled: `pg_stat_replication` empty on both sides

Split-brain guard: before promoting, run `docker rm -f demo3-db-pri` (hard fence) and write a sentinel file (e.g., `/tmp/demo3_primary_fenced.flag`) that `setup_and_run3.sh start` checks before restarting the primary.

Level 4 — Expected vectors

- Dockerfile `RUN` fetches a remote script (pin hash; use `COPY` from a known source)
- Malicious dependency in a requirements file (SBOM + Grype scan)
- Compromised base image (signed base + cosign verification)

WAF Bonus — Expected category

Unicode normalisation bypass works against CRS paranoia-1 for certain keywords; double-URL-encoded paths sometimes slip through Wave 4's patterns; HTTP method-override via query string (`?_method=DELETE`) is not covered by the existing rule set.

Dashboard Injection Bonus — Expected answer

The `cwd=DEMO3_DIR` + `start_new_session=True` in `subprocess.Popen` means the script runs relative to `DEMO3_DIR`. If an attacker can write to `DEMO3_DIR` (e.g., via a separate file-upload vulnerability), they can replace `attack3_web.sh` between trigger calls. Fix: use the absolute path + `os.path.realpath` check, or pin the script to its SHA-256 on startup and verify before `Popen`.